Monero SuperAtomic

Technical Guide

A self-hosted Automated Swap Backend for XMR/BTC atomic swaps, packaged for Umbrel.

Version 0.4.6 • July 2026 • brainchainz

Architecture, design decisions, and the production lessons behind them:
the patched swap engine, the digest proxy, custody and backup, Umbrel packaging,
and the security model, documented end to end.

Contents

1. What This App Is 4

Design philosophy . . . 4

2. System Architecture 5

One container, five processes . . . 5

Trust boundaries . . . 5

The data directory . . . 5

3. The Swap Engine: a Patched Eigen ASB 7

Why the engine embeds a whole Monero wallet . . . 7

The fork . . . 7

The build pipeline . . . 7

4. The Atomic Swap Protocol from the Maker's Chair 9

The happy path . . . 9

The unhappy paths . . . 9

Discovery: rendezvous points and Tor . . . 9

5. Connecting to the Monero Node: the Digest Proxy 10

The problem . . . 10

Rule 1: raw TCP passthrough, never re-serialize . . . 10

Rule 2: the digest handshake must stay on one upstream socket . . . 10

Rule 3: never kill idle client connections . . . 10

6. The Dashboard 11

Stack and shape . . . 11

The authentication model . . . 11

Lifecycle control . . . 11

Polling discipline . . . 11

7. Custody, Backup, and Restore 12

Two independent key domains . . . 12

The archive format . . . 12

The restore pipeline . . . 12

8. Umbrel Packaging Realities 13

The manifest and the store . . . 13

exports.sh runs on the host and is sourced . . . 13

update vs restart . . . 13
The networking truth . . . 13
9. Block Explorers and Activity **14**
The same-host trick . . . 14
Where preferences live . . . 14
Activity, honestly labeled . . . 14
10. Operations **15**
Configuration semantics . . . 15
The Liquidity Planner . . . 15
Logs . . . 15
Deployment pipeline . . . 15
11. Security Posture **16**
Layers, from the outside in . . . 16
Boundaries stated plainly . . . 16
12. Design Decision Index **17**
13. Appendix: API and File Reference **18**
Dashboard API (all under /api, JWT unless noted) . . . 18
Fork RPC additions (engine JSON-RPC on 9944) . . . 18
Repository layout . . . 19

A self-hosted Automated Swap Backend for XMR/BTC atomic swaps, packaged for Umbrel.

Version 0.4.6, July 2026. Written for operators, contributors, and the curious.

1. What This App Is

Monero SuperAtomic turns an Umbrel home server into a **maker** on the XMR/BTC atomic swap network. A maker (in protocol terms, an Automated Swap Backend or ASB) holds Monero, advertises a price, and stands ready to trade XMR for BTC with anyone on the network, without an exchange, an account, or a custodian in the middle. The swap itself is enforced by cryptography: neither side can take the other's funds without releasing their own.

The app is three things layered together:

1. A swap engine: a patched build of the open source Eigen ASB (`eigenwallet/core`, GPLv3), the reference maker implementation.

2. A management dashboard: a Node/Express API and React UI purpose-built for running a maker, covering funding, quoting, swap tracking, withdrawals for both chains, connectivity, configuration, logs, and encrypted backup and restore.

3. Umbrel packaging: compose files, a manifest, and host-side glue that wire the app to the Umbrel Monero node, Bitcoin node, and Electrs server the user already runs, so the maker rides entirely on self-hosted infrastructure.

Design philosophy

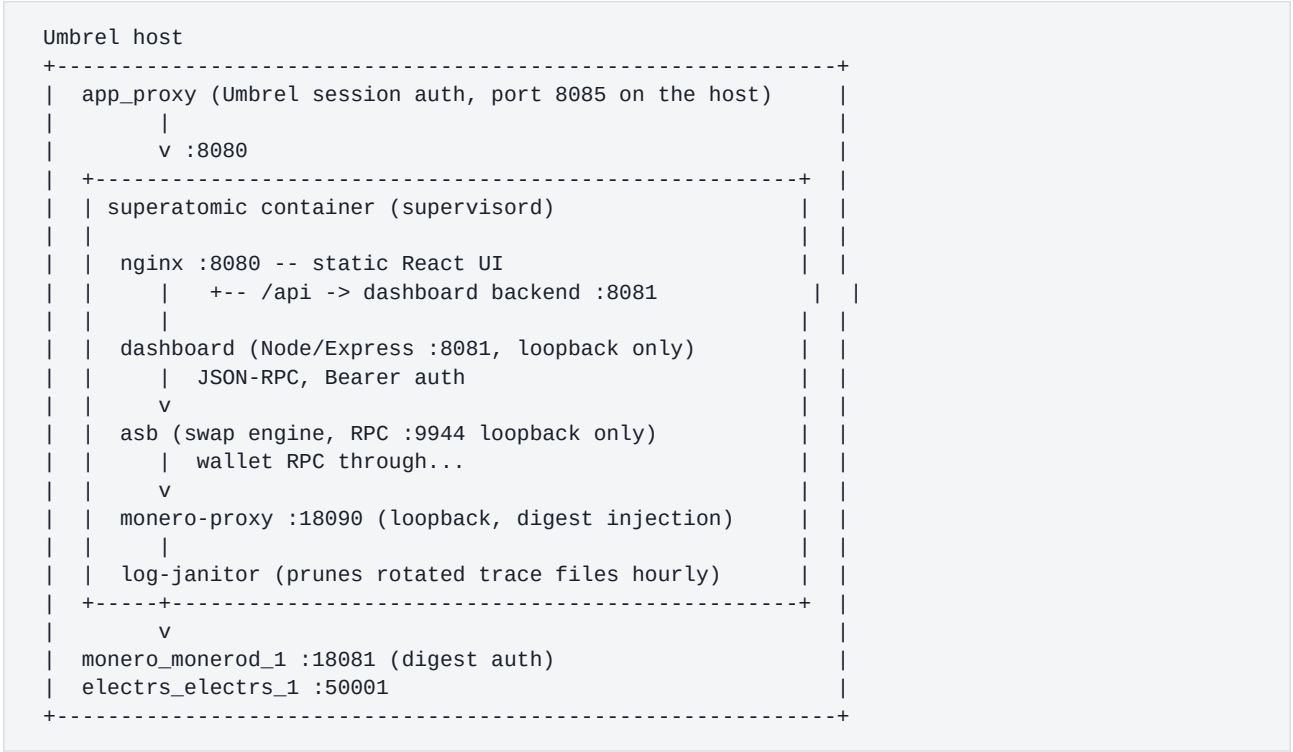
Three principles shaped nearly every decision in this project:

- **Self-hosted all the way down.** The maker talks to your own Monero node, your own Electrs, and (for transaction lookups) your own block explorer apps. Fonts are bundled rather than fetched from Google. The only unavoidable external contact is the swap network itself and the price feeds the engine needs to quote.
- **Privacy by default.** The maker is Tor-only out of the box. The clearnet listener exists but ships disabled, because exposing a home IP to a financial peer-to-peer network is a decision the operator should make deliberately, not a default they discover later.
- **The operator holds the keys, so the app must respect them.** Every action that can move funds requires the dashboard password again, even with a valid session. Backup and restore treat the wallet material as the crown jewels they are. And the app never pretends to more protection than it has: the trust model is documented, including its limits.

2. System Architecture

One container, five processes

The application ships as a single Docker image supervised by `supervisord`. One container was chosen over a multi-container composition because the pieces are tightly coupled (the dashboard controls the swap engine's lifecycle through supervisord) and because Umbrel app slots are simpler to reason about with fewer moving parts.



Process launch order matters and is encoded as supervisord priorities: the log janitor (1), then the Monero proxy (5) so the daemon path exists before the engine wants it, then the ASB (10), then the dashboard (20), then nginx (30).

Trust boundaries

- Everything sensitive binds to loopback inside the container: the ASB's JSON-RPC (9944), the dashboard backend (8081), and the digest proxy (18090). Only nginx (8080, fronted by Umbrel's app_proxy) and the swap engine's peer-to-peer port (9939, published for the optional clearnet mode) face outward.
- The dashboard authenticates to the ASB with a generated RPC password; the browser authenticates to the dashboard with a JWT earned by the user-set password; Umbrel's app_proxy wraps the whole thing in the Umbrel session.

The data directory

Everything stateful lives under one volume, `/data`:

Path	Contents
`seed.pem`	Master seed: Bitcoin wallet, network identity, onion service
`monero/wallets/`	The embedded Monero wallet (cache + keys), independent of seed.pem
`sqlite`	The swap database: every swap's state machine history
`wallet/`	The Bitcoin wallet database
`tor/state/`	Onion service keys, so the maker's address survives restarts
`config.toml`	The engine's configuration (generated once, then user-owned)
`dashboard-password-hash`, `jwt-secret`, `rpc-password`, `rpc-auth`	Dashboard and RPC secrets
`withdrawals.json`, `ui-settings.json`	Withdrawal journal and dashboard preferences
`logs/`	Engine and process logs

3. The Swap Engine: a Patched Eigen ASB

Why the engine embeds a whole Monero wallet

The ASB does not shell out to `monero-wallet-rpc`. It embeds Monero's own wallet2 C++ code in-process through an FFI crate (`monero-sys`). This is why building the engine compiles the Monero codebase, and it is also why the design is safe: exactly one wallet instance ever owns the wallet file, scans the chain, and constructs transactions. Swap transactions and (since 0.4.6) withdrawals go through the same instance, so there is no window in which two views of the wallet can disagree about which outputs are spent.

The fork

The app builds from `github.com/brainchainz/eigenwallet-core`, a public GPLv3 fork of `eigenwallet/core` that stays merged with upstream (it tracks upstream releases; 4.11.4 at the time of writing) and carries four deliberate patches:

1. **Developer tip address** (`swap-env/src/defaults.rs`). Upstream routes the optional maker tip to the eigenwallet developers. This build routes it to the SuperAtomic maintainer. The address is a compiled-in constant, which is why any change requires a full rebuild.
2. **A 1% tip floor** (`swap-asb/src/main.rs`). Upstream defaults the tip to zero and allows any value. This build floors it at 1% of swap volume, in code rather than configuration, so the floor cannot be edited away. This is the app's business model, stated plainly: run the maker, keep your spread, and 1% of volume supports the packaging work. The floor affects only the operator's own economics; it does not change protocol behavior toward counterparties.
3. **Unlocked balance in the RPC** (`swap-controller-api` + `swap/src/asb/rpc/server.rs`). Upstream's `monero_balance` returned only the total. The patch adds the unlocked amount so the dashboard can show an honest Available / Pending split while fresh deposits confirm.
4. `withdraw_xmr` **and** `monero_history` (same files, added in 0.4.6). The wallet layer always had transfer, sweep, and history functions (the engine uses them to lock swap funds); the patch exposes them over the controller RPC. `withdraw_xmr` refuses while any swap is incomplete, because a pending swap's XMR is committed: either already on the shared-key lock address, or promised to a lock transaction the counterparty is waiting on.

A deliberate non-patch is worth recording: **negative spread** (quoting below market to fill fast) was considered and rejected. Upstream's price engine computes the ask with unsigned arithmetic and cannot represent a negative adjustment; supporting it would mean maintaining protocol-adjacent pricing behavior upstream never promised. The rule the project follows: patch local wallet plumbing freely, never patch how the maker behaves toward the network.

The build pipeline

The fork's GitHub Actions workflow (`build-asb-superswap.yml`) compiles the engine into `ghcr.io/brainchainz/asb:<tag>` with a registry layer cache, so the heavy Monero C++ build happens rarely and a typical Rust-only change rebuilds in minutes. The dashboard image then pins the engine **by digest**, not by tag:

```
FROM ghcr.io/brainchainz/asb:4.11.4-1pct-v2@sha256:e61b6f74... AS asb-base
```

Digest pinning means the image that was tested is the image that ships, immune to tag drift and registry-side substitution.

4. The Atomic Swap Protocol from the Maker's Chair

The maker does not need to trust the taker, the taker does not need to trust the maker, and neither needs to trust this software. That is not a slogan; it falls out of the protocol's structure, which is worth understanding as the app's operator.

The happy path

1. Discovery. The maker registers with rendezvous points (see below). A taker's wallet queries them, sees the maker's price and limits, and connects.

2. BTC lock. The taker locks BTC into a script with timelocked escape hatches. The maker cannot touch it yet: redeeming requires key material the taker has not released.

3. XMR lock. Seeing the BTC confirmed, the maker locks XMR to an address whose spend key is the sum of both parties' key shares. From this moment the XMR is not in the maker's wallet and the maker alone cannot move it, whatever software it runs.

4. Redeem. The taker releases the redeem capability. When the maker claims the BTC, the mathematics of the adaptor signature simultaneously reveals the maker's XMR key share to the taker, who now holds both halves and sweeps the XMR. Taking the BTC and surrendering the XMR are one act, not two.

The unhappy paths

- If the maker never locks XMR, the taker refunds their BTC after the first timelock. The maker gains nothing; not locking is indistinguishable from declining.
- If the taker locks BTC and then disappears past their refund window entirely, the maker may eventually claim the BTC through the punish transaction. This protects makers from abandonment. The engine ships a `grant_mercy` command to return funds to an honest taker who was merely offline.
- If a swap is interrupted mid-flight (a crash, a power cut), the swap database holds the state needed to resume or recover. This is why the database is part of the backup, and why the app nags about backups at all.

The practical consequence for this app's design: features like XMR withdrawal are safe to add because **no local modification can break atomicity**. The withdrawal guard (refuse during active swaps) protects the operator from failing their own swaps, not the counterparty from theft; theft is prevented by the protocol.

Discovery: rendezvous points and Tor

Makers are found through rendezvous servers. The app ships the current eigenwallet set: four operators, each listed twice, once as a clearnet WSS address and once as a Tor onion service. The duplication matters for this app specifically: a Tor-only maker dials the onion addresses natively, which is far more reliable than tunneling a clearnet connection through Tor exits. (An earlier release shipped a stale rendezvous list inherited from the pre-eigenwallet era; most of those hosts were dead, which taught the lesson that this list must track upstream.)

The maker's own reachability is its Tor onion service, published with 10 introduction points (raised from the default 5) to give takers more entry paths. "Disconnected" in a taker's UI between trades is normal: takers connect on demand to pull a quote, then drop the connection.

5. Connecting to the Monero Node: the Digest Proxy

This small component earned its complexity the hard way, and its three design rules each encode a production incident.

The problem

Umbrel's monerod enforces HTTP digest authentication on **every** RPC port, including the restricted one. The ASB's Monero connection layer stores nodes as plain scheme/host/port and cannot send credentials. Without help, the engine simply cannot talk to the node it sits next to.

The fix is a tiny in-container proxy: the engine talks plain HTTP to `127.0.0.1:18090`, and the proxy injects the digest handshake toward `monero_monerod_1:18081`, resolving the node by Docker DNS name so IP churn cannot break it.

Rule 1: raw TCP passthrough, never re-serialize

The wallet's block scanner is monerod's own epee HTTP client, and it is extremely strict about response bytes. A conventional HTTP proxy (Node's `http` module, or anything similar) re-serializes responses: it adds Date and Connection headers and lowercases header names. The epee client rejects the result (`http_stream_filter::parse_cached_header`) and the wallet silently scans zero blocks. The symptom was a balance stuck at zero while the funds sat on chain.

So the proxy operates at the TCP level. It parses just enough of each request to inject an Authorization header, then streams monerod's response bytes back **unmodified**, byte-identical to what the daemon sent.

Rule 2: the digest handshake must stay on one upstream socket

monerod issues its digest nonce **per connection**. An unauthenticated probe on one socket followed by an authenticated retry on a fresh socket presents a nonce the daemon has never seen on that connection, and every request 401s. The working sequence, per client request: open one upstream connection, send unauthenticated, read the 401 challenge, answer on the **same** socket, stream the response back. This mirrors what `curl --digest` does with keep-alive.

Rule 3: never kill idle client connections

The engine's HTTP client pools its connection to the proxy and reuses it minutes later. An earlier proxy version destroyed idle client sockets after two minutes; the engine would then wake, use the corpse, fail with a closed-channel error, reconnect, succeed, idle, and die again. The visible effect was quotes failing on **alternate attempts**, which takers experienced as the maker intermittently having "no funds available." The fix: no idle timeout at all on client connections, with TCP keepalive to reap genuinely dead peers. After the fix, quote computation went from roughly 50% failure to zero.

6. The Dashboard

Stack and shape

- **Backend**: Node/Express, a deliberately thin translation layer between the browser and the engine's JSON-RPC. It holds no wallet logic of its own; it authenticates, validates, converts units (satoshi and piconero at the RPC boundary, whole BTC and XMR in the UI), and journals what the engine cannot.
- **Frontend**: React with Vite, a single-page app of nine tabs: Overview, Funding, Planner, Withdraw, Network, Config, Backup, Logs, and How-To.
- **nginx** serves the built UI and reverse-proxies `/api`, with a strict Content-Security-Policy (the app is fully self-contained; fonts are bundled via Fontsource precisely so the CSP can say so and no page view pings Google).

The authentication model

Three layers, each with a distinct job:

1. **Umbrel's app_proxy session**: keeps strangers on the network out.
2. **A dashboard password the user sets on first run**: the app refuses to invent one. An early design generated a password host-side; it was replaced because a password the user chose is one the user can remember, rotate, and reason about. The hash is bcrypt (cost 12), stored in the data directory.
3. **A JWT (7 days)** earned by that password, held in the browser.

Two hardening rules sit on top:

- **Anything that can move or expose funds re-confirms the password**, session or not: withdrawals on both chains, seed reveal, backup download, restore. A stolen or forgotten-open browser tab must not be able to drain the wallet.
- **Changing the password rotates the JWT secret**, killing every existing session immediately. A password change usually means the operator suspects a leak; old tokens must die now, not in a week.

Lifecycle control

The backend can stop, start, and restart the engine through supervisord. This powers "Save and Restart" in the Config tab (the engine only reads its configuration at startup) and the restore flow (wallet files must not be overwritten under a live process). An early version of the Config tab wrote the file and never restarted; the lesson, now encoded in the UI, is that a settings screen must either apply settings or say loudly that it has not.

Polling discipline

Every live panel polls on its own timer with error handling that keeps the loop alive through engine restarts, and the API helper the panels share has a stable identity across renders. Both halves were lessons: an unstable helper identity made a background status poll silently re-trigger data loads that wiped in-progress form edits, and unguarded polls died permanently on the first failed request during a restart.

7. Custody, Backup, and Restore

Two independent key domains

This is the single most important custody fact about the app, and it is surprising:

- `seed.pem` derives the Bitcoin wallet, the maker's network identity (peer ID), and the Tor onion service key.
- **The Monero wallet is independent.** It is generated separately and is not derived from `seed.pem`. Backing up one does not back up the other.

A "backup" that captured only `seed.pem` would silently omit the asset the maker exists to hold. The full backup therefore packages: `seed.pem`, the Monero wallet files, the swap database (required to recover a swap interrupted mid-flight), the Bitcoin wallet database, the Tor onion key, the withdrawal journal, and dashboard preferences.

The archive format

The backup is a tar.gz, optionally encrypted client-side of the download with a passphrase. The encrypted form is:

```
"SABK1" | salt (16 bytes) | IV (12 bytes) | GCM tag (16 bytes) | ciphertext
```

Key derivation is scrypt; the cipher is AES-256-GCM, so tampering is detected, not just garbled. A wrong passphrase fails cleanly before a single live file is touched.

The restore pipeline

Restore is the mirror image, engineered defensively because its failure mode is overwriting live wallets:

1. Password re-confirmation (credentials travel base64-encoded in headers, since the body is the raw archive).
2. Decrypt if the SABK1 magic is present; then verify the result is actually gzip.
3. **List the archive before extracting.** Every entry must match a whitelist of known top-level names; anything else, any absolute path, any `..`, and any symlink or hardlink entry (which could redirect later writes outside the data directory) aborts the restore untouched.
4. Stop the engine, extract, start the engine. The wallet rescans and the maker resumes with its original identity, onion address, and funds.

The restore path is validated end to end in the field: a full backup, uninstall, reinstall, restore cycle preserved the wallet balance and identity. (It also surfaced a bug: nginx's default 1 MB body limit rejected the upload with HTTP 413. The limit is now 600 MB with request buffering off.)

One boundary is intentional: the dashboard password is **not** in the backup. A backup file that could unlock its own restore would collapse two factors into one.

8. Umbrel Packaging Realities

Packaging for Umbrel has rules, several of which are undocumented and were learned by breaking them. They are recorded here so nobody breaks them twice.

The manifest and the store

- The app directory name in the community store **must equal** the manifest `id`.
- The manifest `port` is the host port and must be unique across all installed apps.
- `dependencies` lists the Umbrel apps this one needs: `monero`, `bitcoin`, `electrs`.

exports.sh runs on the host and is sourced

Umbrel **sources** `exports.sh` into the environment it uses to interpolate the compose file. Two sharp edges follow:

- **Variable names collide globally.** A variable literally named `JWT_SECRET` overrode the app_proxy's own `JWT_SECRET` and broke Umbrel session validation for the whole app (the symptom was a blank page and a signature error). Every generated-secret variable is therefore namespaced `GEN_*`.
- The script generates `config.toml` **only when absent**, so user edits survive restarts, and regeneration is achieved by deleting the file and restarting.

update vs restart

Two umbreld operations do different halves of a deployment:

- `apps.update` pulls the new image and recreates containers, but does not re-run `exports.sh`.
- `apps.restart` re-runs `exports.sh` (regenerating config if absent), but does not pull images.

A release that changes both the image and the host-side config needs both, in that order.

The networking truth

Umbrel's compose overlay maps every app's `default` network **onto the shared** `umbrel_main_network`. An app-defined private subnet is silently never created, and consequences cascade:

- A static `ipv4_address` on the "private" subnet is out of range for the real network; the attachment silently drops.
- An nginx allow-list keyed to the private subnet can never match a real client. (Release 0.4.5 shipped exactly that allow-list and locked out the app_proxy itself with 403s until it was reverted the same hour.)
- Inter-app reachability on the shared network is inherent to the platform. The app's protection is therefore its **own authentication layers**, not network ACLs. This is the platform's actual trust model, and the app now documents it instead of fighting it.

9. Block Explorers and Activity

The same-host trick

Users reach the dashboard by many names: `umbrel.local`, a Tailscale IP, a LAN IP. Any hardcoded explorer URL would be wrong for someone. The solution is to derive the explorer origin **from however the user is reaching the dashboard right now**:

```
http://<current hostname>:3006/tx/<txid>   (Bitcoin: the Umbrel mempool app)
http://<current hostname>:3008/tx/<txid>   (Monero: the MoneroSpace app)
```

Whatever hostname loaded the page also reaches the explorer apps, because they live on the same machine. The moving-target problem (Tailscale IPs differ per user and session) evaporates without configuration. Manual override URLs exist in the Config tab for Tor access (each app has its own onion address, so same-host-different-port does not apply) and for anyone preferring a public explorer, with the privacy note that a public explorer learns which transactions you look up. Self-hosted is the default for a reason.

Where preferences live

Explorer URLs cannot go in `config.toml`: the engine's parser rejects unknown fields outright (a good property; it means typos fail loudly), and would refuse to boot. Dashboard-only preferences therefore live in their own file, `ui-settings.json`, with a small API of their own.

Activity, honestly labeled

- **Monero activity** comes straight from the embedded wallet's transaction history (exposed by the 0.4.6 fork patch): every deposit and withdrawal, with confirmations and explorer links.
- **Bitcoin activity** is assembled from the two sources that exist: swap redeem and punish transactions from the swap database, and a local journal of withdrawals the dashboard itself broadcast. The engine exposes no full Bitcoin wallet history, so direct deposits to the internal wallet do not appear, and the UI says so rather than implying completeness.

10. Operations

Configuration semantics

The Config tab edits `config.toml` through the API. Percentages display as whole numbers (a 2% spread is "2", not "0.02") and convert to fractions on save, because a UI that makes 100x errors easy is a UI that will eventually cause one. Saving writes the file and **restarts the engine**, with the banner narrating both steps; quotes resume after the wallet re-syncs, typically within a minute.

The Liquidity Planner

The Planner answers the operator's real question: "at my balance and a given spread, what do I earn, and what should my limits be?" It derives the market rate from the engine's live quote (ask price divided by one plus the current spread), lets the operator slide the spread, and computes ask price, full-liquidity BTC value, spread earnings, and the 1% tip. An "Apply to Config" button carries the chosen spread and suggested maximum into the Config form, deliberately unsaved, so the operator reviews before anything takes effect.

Logs

The engine writes hourly-rotated trace files, and the libp2p traces alone run to roughly 1.8 GB per day if never pruned (a production box reached 1.7 GB before this was caught). A janitor process deletes libp2p traces older than 3 hours and general traces older than 7 days. The Logs tab reads only the last 256 KB of the current log per refresh, so reads stay cheap as files grow.

Deployment pipeline

A release moves through fixed stages:

1. Engine changes (if any): commit to the fork, CI builds `ghcr.io/brainchainz/asb:<tag>`, note the digest.
2. Dashboard image: pin the engine digest in the Dockerfile, build for `linux/amd64`, push `sirjamzalot/monero-superatomic:<version>`.
3. Store: bump `version` and the compose image tag, write release notes, commit, push.
4. Box: update the store clone, `apps.update` (plus `apps.restart` if host-side files changed), then verify: processes running, balance intact, quote computing, dashboard reachable.

Every deploy ends with verification against the live box, because the 0.4.5 networking incident proved that "it built" and "users can reach it" are different facts.

11. Security Posture

Layers, from the outside in

Layer	Protects against
Umbrel app_proxy session	Anyone on the network reaching the UI at all
Dashboard password (bcrypt, user-set, first run)	Anyone past the Umbrel session
JWT with rotate-on-password-change	Stale or stolen sessions outliving a password change
Per-action password confirmation	A hijacked session moving funds
Engine RPC on loopback with Bearer auth	Anything in the container short of root
Digest proxy on loopback	Other containers borrowing the node credentials
Restore whitelist + link rejection	Malicious backup archives writing outside `/data`
Digest-pinned images	Registry-side image substitution
Rate limiting (120 req/min on the API)	Brute force at the HTTP layer
CSP + bundled fonts	Third-party content injection and passive tracking

Boundaries stated plainly

- **The shared Umbrel network is flat.** Other app containers can reach this app's nginx. Authentication, not network topology, is the protection, and the narrow first-run window (before a password exists) is inherent to the platform; the How-To instructs setting the password first.
- **A compromised host is game over**, as with any self-hosted wallet. Secrets are files with restrictive modes in the app's data directory; root on the box reads them. The app's job is to not widen that surface.
- **The protocol does not depend on this software's honesty** (section 4), which cuts both ways: the app cannot cheat counterparties, and counterparties running modified software cannot cheat the app.

The UI carries a plain-language disclaimer in the How-To tab: this is an independent packaging of the open source Eigen ASB; the packagers are not the protocol's developers, custody no funds, and disclaim liability for losses; the operator owns their configuration, backups, and funds.

12. Design Decision Index

A compressed record of the choices and their reasons.

Decision	Reason
Tor-only by default, clearnet opt-in	A home IP on a financial P2P network is a deliberate choice, never a default
User sets the dashboard password on first run	Generated passwords get lost; owned passwords get managed
Password re-confirmation on every fund-moving action	Sessions get stolen; the password should not travel with them
JWT secret rotates on password change	A password change means "lock everyone out now"
1% tip floor compiled into the binary	The business model should not be a config edit away from zero
Fork tracks upstream releases closely	Protocol fixes and version floors arrive on upstream's schedule
No negative spread patch	Local wallet plumbing is fair game; network-facing pricing behavior is upstream's
XMR withdrawal via the engine, not monero-wallet-rpc	Exactly one process may own a wallet's outputs; a second spender invites self-conflict
Withdrawals refused during active swaps	Committed XMR is not the operator's to move until the swap resolves
Raw TCP digest proxy	The epee client rejects any re-serialized response
No idle timeout on proxy client connections	The engine pools connections; killing them made alternate quotes fail
Single container under supervisord	The dashboard must control the engine's lifecycle
Engine image pinned by digest	The tested image is the shipped image
Backup includes the swap database	Stuck swaps are recovered from state, not from seeds
Dashboard password excluded from backup	The backup must not unlock its own restore
Restore whitelists entries and rejects links	The failure mode is writing outside `/data` over live wallets
Explorer URLs derived from the current hostname	Tailscale and LAN addresses differ per user; the browser already knows the right one
UI preferences in `ui-settings.json`, never `config.toml`	The engine rejects unknown config fields, and should
Percentages as whole numbers in the UI	2 vs 0.02 is a 100x error waiting to happen
Fonts bundled, CSP strict	A privacy app's dashboard should not ping Google per page view
Whole-percent UI, fraction on disk	The engine's native unit and the human's native unit differ; convert at the boundary
No network ACLs inside Umbrel	The platform's flat network makes them theater; honest auth beats fake isolation

13. Appendix: API and File Reference

Dashboard API (all under `/api`, JWT unless noted)

Endpoint	Method	Purpose
`/auth/status`	GET	Is a password set yet (no auth)
`/auth/setup`	POST	First-run password creation (no auth, once)
`/auth/login`	POST	Exchange password for JWT (no auth)
`/auth/check`	GET	Validate token
`/auth/change`	POST	Change password, rotate JWT secret
`/status`	GET	Engine connectivity
`/balances`	GET	XMR (total/unlocked/pending) + BTC
`/deposit`	GET	Monero funding address
`/quote`	GET	Current advertised offer
`/swaps`	GET	Swap list with states and txids
`/connectivity`	GET	Peer ID, connections, rendezvous, onion
`/withdraw`	POST	BTC out (password re-confirmed)
`/withdraw-xmr`	POST	XMR out (password re-confirmed, refused mid-swap)
`/monero-history`	GET	Wallet deposits and withdrawals
`/activity`	GET	BTC activity (journal + swap txs)
`/config`	GET/POST	Read and write config.toml
`/restart`	POST	Restart the engine
`/logs`	GET	Tail of the engine log
`/seeds/:type`	POST	Reveal Monero seed / BTC descriptor (password re-confirmed)
`/backup`	POST	Download encrypted archive (password re-confirmed)
`/restore`	POST	Upload and apply archive (password re-confirmed)
`/ui-settings`	GET/POST	Explorer URLs and preferences

Fork RPC additions (engine JSON-RPC on 9944)

Method	Added in	Purpose
`monero_balance.unlocked`	0.3.x	Available/pending split

Method	Added in	Purpose
`withdraw_xmr(address, piconero?)`	0.4.6	Transfer or sweep; guarded against active swaps
`monero_history()`	0.4.6	Wallet transaction list

Repository layout

```
eigen-asb-dashboard/          The image source
+-- Dockerfile                Engine (by digest) + dashboard + nginx + supervisord
+-- backend/server.js         The entire API
+-- frontend/src/             React app (one component per tab, shared Txid/explorers)
+-- nginx/nginx.conf          Static serving, /api proxy, CSP, 600 MB restore limit
+-- scripts/
|   +-- monero-proxy.mjs      The raw TCP digest proxy
|   +-- log-janitor.sh        Hourly trace pruning
|   +-- entrypoint.sh         Volume permissions, then supervisord
|   +-- start-asb.sh          Engine launch with RPC auth
+-- supervisord.conf          Process tree and priorities

Monero-Superbrain/            The community app store
+-- brainchainz-monero-superatomic/
    +-- umbrel-app.yml        Manifest: id, version, port, dependencies
    +-- docker-compose.yml    app_proxy + server, shared network
    +-- exports.sh            Host-side: node credentials, secret generation, config.toml

github.com/brainchainz/eigenwallet-core    The engine fork (GPLv3)
```

Monero SuperAtomic is an independent packaging of the open source Eigen ASB with a management dashboard. It is provided as is, without warranty of any kind. This guide describes version 0.4.6.